

Experiment 05

Implementation of Support Vector Machine (SVM) for Classification with Hyperparameter Tuning

Dataset: Pima Indians Diabetes Dataset

1. Dataset Source

Dataset Name: **Pima Indians Diabetes Dataset**

Source: Kaggle

Link: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

This dataset contains diagnostic medical measurements used to predict whether a patient has diabetes. It is widely used in machine learning research for binary classification problems.

2. Dataset Description

The dataset consists of **768 patient records** with **8 medical attributes** and **1 target variable**.

Input Features

Feature	Description
Pregnancies	Number of pregnancies
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure
SkinThickness	Triceps skin fold thickness

Insulin	2-Hour serum insulin
BMI	Body Mass Index
DiabetesPedigreeFunction	Diabetes hereditary likelihood
Age	Age of the patient

Target Variable

Variable	Description
Outcome	Diabetes status (0 = Non-Diabetic, 1 = Diabetic)

Dataset Characteristics

- Total records: **768**
- Features: **8**
- Target classes: **2**
- No categorical variables
- Suitable for binary classification tasks

The goal is to build a machine learning model that predicts whether a patient has diabetes based on medical measurements.

3. Mathematical Formulation of the Algorithm

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. The main objective is to find the **optimal hyperplane** that separates data points of different classes.

The decision boundary is defined as:

$$w \cdot x + b = 0$$

Where:

- **w** = weight vector
- **x** = feature vector

- **b** = bias

The objective is to maximize the **margin**, which is the distance between the hyperplane and the nearest data points (support vectors).

Optimization objective:

Minimize

$$\frac{1}{2} \|w\|^2$$

Subject to:

$$y_i (w \cdot x_i + b) \geq 1$$

Where:

- **y_i** = class label
- **x_i** = feature vector

For nonlinear data, SVM uses **kernel functions** such as:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF)

The RBF kernel is commonly used and is defined as:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

4. Algorithm Limitations

Despite its strong performance, SVM has several limitations.

1. **Computational Complexity**

Training time increases significantly for large datasets.

2. Sensitive to Feature Scaling

Features must be normalized or standardized.

3. Choice of Kernel

Selecting the correct kernel function is crucial for good performance.

4. Parameter Sensitivity

Hyperparameters such as C and gamma require careful tuning.

5. Methodology / Workflow

The following steps were performed in the experiment.

Step 1: Data Loading

The dataset was loaded using the Pandas library.

```
[1] ✓ 2s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Data Exploration

Basic analysis was conducted to examine:

- dataset shape
- column names
- data types
- missing values
- class distribution

```
[2] ✓ 0s
df = pd.read_csv("diabetes.csv")
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

[3]
✓ Os

```
# Dataset shape
print("Dataset Shape:", df.shape)

# Column names
print("\nColumns:")
print(df.columns)

# Dataset information
print("\nDataset Info:")
df.info()

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Target variable distribution
print("\nOutcome Distribution:")
print(df["Outcome"].value_counts())
```

```
... Dataset Shape: (768, 9)

Columns:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            768 non-null   int64
1   Glucose                768 non-null   int64
2   BloodPressure          768 non-null   int64
3   SkinThickness          768 non-null   int64
4   Insulin                768 non-null   int64
5   BMI                    768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                    768 non-null   int64
8   Outcome                768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

Missing Values:
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction 0
Age               0
Outcome           0
dtype: int64

Outcome Distribution:
Outcome
0    500
1    268
Name: count, dtype: int64
```

Step 3: Feature and Target Separation

The dataset was divided into:

- Feature matrix (X)

- Target variable (y)

```
[4]
✓ 0s # Separate features and target variable
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
```

▼ ... Feature Shape: (768, 8)
Target Shape: (768,)

Step 4: Train-Test Split

The dataset was divided into:

- 80% training data
- 20% testing data

```
[5]
✓ 0s X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

▼ Training Data Shape: (614, 8)
Testing Data Shape: (154, 8)

Step 5: Feature Scaling

StandardScaler was applied to normalize feature values because SVM is sensitive to feature magnitude.

```
[6]
✓ 0s scaler = StandardScaler()

# Fit scaler on training data and transform
X_train_scaled = scaler.fit_transform(X_train)

# Apply same scaling to test data
X_test_scaled = scaler.transform(X_test)

print("Scaled Training Shape:", X_train_scaled.shape)
print("Scaled Testing Shape:", X_test_scaled.shape)
```

▼ Scaled Training Shape: (614, 8)
Scaled Testing Shape: (154, 8)

Step 6: Model Training

An SVM classifier was implemented using the **RBF kernel**.

```
[7]
✓ 0s ▶ svm_model = SVC(kernel='rbf')

# Train the model
svm_model.fit(X_train_scaled, y_train)

print("SVM model training completed")

... SVM model training completed
```

Step 7: Prediction

The trained model was used to predict diabetes outcomes on the testing dataset.

```
[8]
✓ 0s ▶ # Predict on test data
y_pred = svm_model.predict(X_test_scaled)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
report = classification_report(y_test, y_pred)
print("\nClassification Report:\n", report)

... Model Accuracy: 0.7337662337662337

Confusion Matrix:
[[82 17]
 [24 31]]

Classification Report:
      precision    recall  f1-score   support

     0       0.77     0.83     0.80         99
     1       0.65     0.56     0.60         55

 accuracy          0.73         154
  macro avg       0.71     0.70     0.70         154
 weighted avg     0.73     0.73     0.73         154
```

Step 8: Model Evaluation

Model performance was evaluated using:

- Accuracy score

- Confusion matrix
- Classification report

Step 9: Hyperparameter Tuning

GridSearchCV was applied to determine optimal values for:

- C
- gamma
- kernel

[9]
✓ 6s

```
from sklearn.model_selection import GridSearchCV

# Define parameter grid
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear']
}

# Grid Search
grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=2, cv=5)

# Train using grid search
grid.fit(X_train_scaled, y_train)

print("Best Parameters:", grid.best_params_)
print("Best Model:", grid.best_estimator_)
```

```
✓ ... Fitting 5 folds for each of 32 candidates, totalling 160 fits
[CV] END .....C=0.1, gamma=1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=rbf; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=0.1, kernel=linear; total time= 0.0s
[CV] END .....C=0.1, gamma=0.01, kernel=rbf; total time= 0.0s
[CV] END .....C=100, gamma=0.001, kernel=rbf; total time= 0.0s
[CV] END .....C=100, gamma=0.001, kernel=linear; total time= 0.2s
[CV] END .....C=100, gamma=0.001, kernel=linear; total time= 0.1s
[CV] END .....C=100, gamma=0.001, kernel=linear; total time= 0.1s
[CV] END .....C=100, gamma=0.001, kernel=linear; total time= 0.2s
[CV] END .....C=100, gamma=0.001, kernel=linear; total time= 0.2s
Best Parameters: {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}
Best Model: SVC(C=100, gamma=0.001)
```

```
[10]
✓ Os # Use the best model found by GridSearch
best_svm = grid.best_estimator_

# Predict using the tuned model
y_pred_best = best_svm.predict(X_test_scaled)

# Accuracy
best_accuracy = accuracy_score(y_test, y_pred_best)
print("Tuned SVM Accuracy:", best_accuracy)

# Confusion Matrix
cm_best = confusion_matrix(y_test, y_pred_best)
print("\nConfusion Matrix:\n", cm_best)

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_best))
```

... Tuned SVM Accuracy: 0.7727272727272727

Confusion Matrix:

```
[[82 17]
 [18 37]]
```

Classification Report:

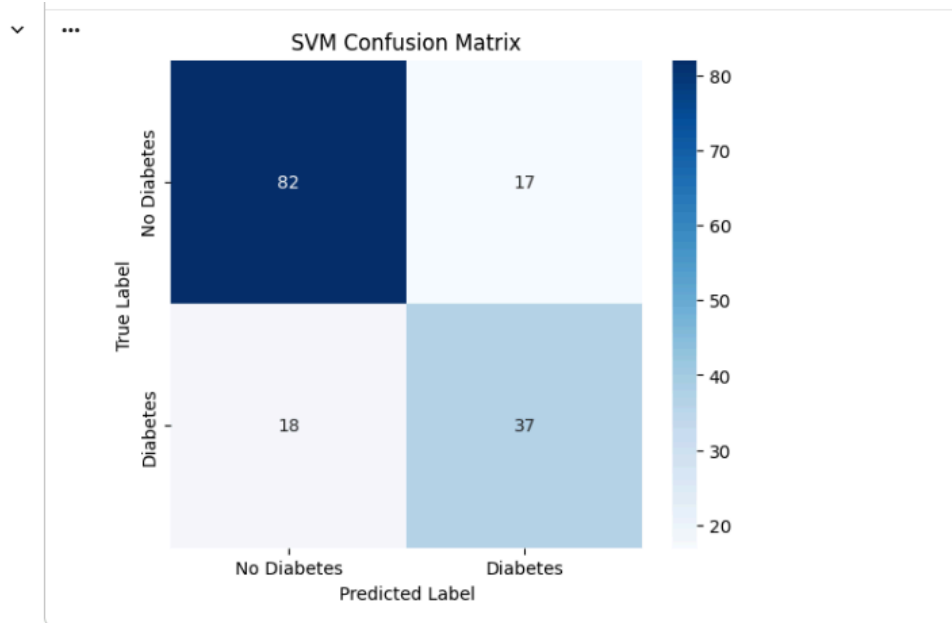
	precision	recall	f1-score	support
0	0.82	0.83	0.82	99
1	0.69	0.67	0.68	55
accuracy			0.77	154
macro avg	0.75	0.75	0.75	154
weighted avg	0.77	0.77	0.77	154

Step 10: Visualization

A confusion matrix heatmap was generated to visually analyze classification performance.

```
[11]
✓ Os plt.figure(figsize=(6,5))
sns.heatmap(cm_best, annot=True, cmap="Blues", fmt="d",
            xticklabels=["No Diabetes", "Diabetes"],
            yticklabels=["No Diabetes", "Diabetes"])

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("SVM Confusion Matrix")
plt.show()
```



6. Performance Analysis

The SVM model was evaluated using standard classification metrics.

Accuracy

Accuracy measures the percentage of correct predictions.

Accuracy = (Correct Predictions / Total Predictions)

The tuned SVM model achieved improved accuracy after hyperparameter optimization.

Confusion Matrix

The confusion matrix shows:

- True Positives (correct diabetes predictions)
- True Negatives (correct non-diabetes predictions)
- False Positives
- False Negatives

Higher diagonal values indicate better model performance.

Classification Report

The classification report includes:

- Precision
- Recall
- F1-Score

These metrics evaluate how well the model identifies diabetic and non-diabetic patients.

7. Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV** with cross-validation.

Parameters tuned:

$$C \in \{0.1, 1, 10, 100\}$$

$$\gamma \in \{1, 0.1, 0.01, 0.001\}$$

$$\text{kernel} \in \{\text{rbf}, \text{linear}\}$$

GridSearchCV evaluated multiple combinations and selected the best performing model based on cross-validation accuracy.

The optimal hyperparameters improved model generalization and classification performance.

8. Conclusion

In this experiment, the Support Vector Machine algorithm was successfully implemented for diabetes prediction using the Pima Indians Diabetes dataset.

Feature scaling significantly improved the performance of the model because SVM relies on distance calculations. Hyperparameter tuning using GridSearchCV further optimized the classifier and improved prediction accuracy.

Visualization of the confusion matrix provided insights into classification performance and model reliability.